

Contents

- [Project 5 report](#)
- [step 3 - extract camera RGB values for the gray-patch](#)
- [step 4 - extract measured gray-patch Y values](#)
- [step 5 - plot measured gray-patch Ys vs. camera RGB values](#)
- [step 6 - linearize camera's RGB response w.r.t relative luminance \(Y\)](#)
- [step 7 - plot measured gray-patch Ys vs linearized camera gray-patch RGBs](#)
- [step 8 - visualize the pre- and post- linearization ColorChecker patches](#)
- [step 9 - derive a matrix to transform from linearized RGBs to XYZs](#)
- [step 10 - use the matrix to estimate XYZ values from linearized RGBs](#)
- [step 11 - evaluate the accuracy of the camera color model](#)
- [step 12 - compensate for interactions and non-linearities](#)
- [step 13 - estimate XYZs from the extended matrix and RS representation](#)
- [step 14 - evaluate the extended camera model by using deltaE](#)
- [step 15 - save the extended camera model](#)
- [step 16 - create camRGB2XYZ function to convert camera RGBs to estimated XYZs](#)
- [step 17 - visualize camera model and camRGB2XYZ function](#)

Project 5 report

Zhen Lai 11/02/2024

This project characterizes the color response of an iPhone SE camera using a photographed ColorChecker and corresponding ColorMunki measurements. The resulting model converts camera RGB values into estimated CIE XYZ values.

- Steps 3 and 4 assemble the camera RGB values and measured XYZ/CIELAB reference values for the 24 ColorChecker patches.
- Steps 5 through 8 use the six neutral patches to visualize the camera's nonlinear tonal response, fit a cubic linearization curve for each RGB channel, and compare the original and linearized patch values.
- Steps 9 through 11 derive a baseline 3-by-3 linear RGB-to-XYZ matrix and evaluate its accuracy using CIE 1976 Delta E*ab.
- Steps 12 through 14 construct an extended 3-by-11 model containing RGB interaction, squared, and constant terms, then evaluate its color error.
- Steps 15 and 16 save the extended model and demonstrate the reusable `camRGB2XYZ` conversion function.
- Step 17 chromatically adapts the measured and camera-estimated XYZ values from D50 to D65 and renders both sets as sRGB ColorChecker images for a visual comparison.

```
repo_dir = fileparts(fileparts(fileparts(fileparts(mfilename('fullpath')))));
addpath(fullfile(repo_dir, 'shared'));
paths = imgs351Paths(repo_dir);
addpath(paths.p03.code, paths.p04.code, paths.p05.code);
```

step 3 - extract camera RGB values for the gray-patch

use PhotoShop to find the average RGB values for each patch in the chart

```
cam_RGBs = [89 53 39; 215 158 138; 79 103 147; 62 83 42; 119 114 170; 107 195 179;
            226 125 53; 32 53 146; 205 78 89; 51 22 66; 170 205 77; 236 177 73;
            3 3 103; 53 124 56; 163 23 32; 250 223 82; 182 63 131; 2 100 145;
            234 233 229; 197 196 194; 137 137 137; 86 88 87; 39 43 44; 5 6 8;];
cam_RGBs = cam_RGBs'

% normalize the RGB values by dividing the values by 255
cam_rgbs = cam_RGBs ./ 255

% extract the gray patches and flip their order from black to white
cam_gray_rgbs = fliplr(cam_rgbs(:, 19:end))
```

```
cam_RGBs =

Columns 1 through 13

    89    215     79     62    119    107    226     32    205     51    170    236     3
    53    158    103     83    114    195    125     53     78     22    205    177     3
    39    138    147     42    170    179     53    146     89     66     77     73    103

Columns 14 through 24

    53    163    250    182     2    234    197    137     86     39     5
   124     23    223     63    100    233    196    137     88     43     6
    56     32     82    131    145    229    194    137     87     44     8

cam_rgbs =
```

Columns 1 through 7

0.3490	0.8431	0.3098	0.2431	0.4667	0.4196	0.8863
0.2078	0.6196	0.4039	0.3255	0.4471	0.7647	0.4902
0.1529	0.5412	0.5765	0.1647	0.6667	0.7020	0.2078

Columns 8 through 14

0.1255	0.8039	0.2000	0.6667	0.9255	0.0118	0.2078
0.2078	0.3059	0.0863	0.8039	0.6941	0.0118	0.4863
0.5725	0.3490	0.2588	0.3020	0.2863	0.4039	0.2196

Columns 15 through 21

0.6392	0.9804	0.7137	0.0078	0.9176	0.7725	0.5373
0.0902	0.8745	0.2471	0.3922	0.9137	0.7686	0.5373
0.1255	0.3216	0.5137	0.5686	0.8980	0.7608	0.5373

Columns 22 through 24

0.3373	0.1529	0.0196
0.3451	0.1686	0.0235
0.3412	0.1725	0.0314

cam_gray_rgbs =

0.0196	0.1529	0.3373	0.5373	0.7725	0.9176
0.0235	0.1686	0.3451	0.5373	0.7686	0.9137
0.0314	0.1725	0.3412	0.5373	0.7608	0.8980

step 4 - extract measured gray-patch Y values

```
% load the ColorMunki measured XYZ and Lab values of the ColorChecker chart
% into 2 arrays for XYZ and Lab values respectively
munki_data = importdata(fullfile(paths.p05.data, ...
    'munki_CC_XYZs_Labs.txt'));
munki_XYZs = munki_data(:, 2:4);
munki_Labs = munki_data(:, 5:end)';

% extract the gray-patch Y values, normalizing the values and flip the order
munki_gray_Ys = fliplr(munki_XYZs(2, 19:end) ./ 100)
```

munki_gray_Ys =

0.0307	0.0858	0.1889	0.3534	0.5674	0.8868
--------	--------	--------	--------	--------	--------

step 5 - plot measured gray-patch Ys vs. camera RGB values

```
% assign each row to corresponding RGB channels
r = 1;
g = 2;
b = 3;

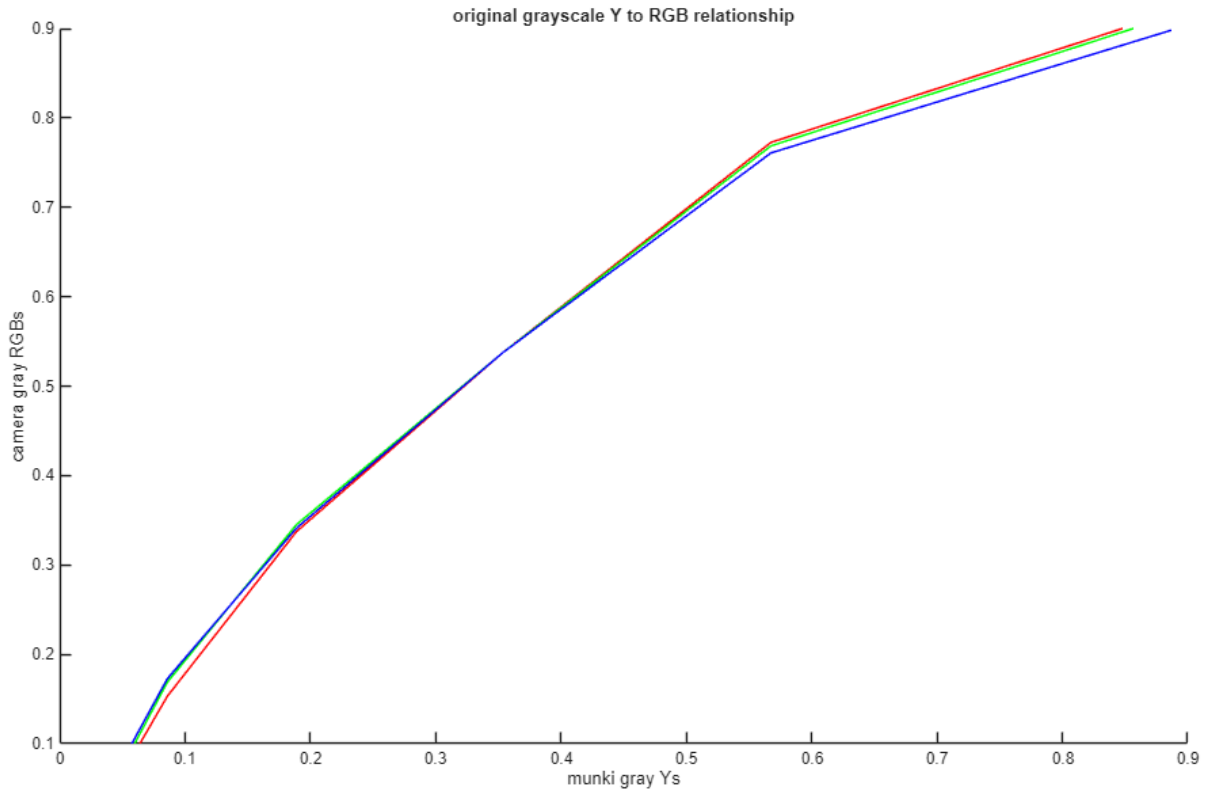
% visualize camera's TTFs
figure;
hold on

% Ys vs Rs
plot(munki_gray_Ys, cam_gray_rgbs(r, :), "Color", 'r');
% Ys vs Gs
plot(munki_gray_Ys, cam_gray_rgbs(g, :), "Color", 'g');
% Ys vs Bs
plot(munki_gray_Ys, cam_gray_rgbs(b, :), "Color", 'b');

% adjust font size
fontsize(gcf, scale=0.9)

hold off

xlim([0 0.9])
ylim([0.1 0.9])
xticks(0:0.1:0.9)
yticks(0:0.1:0.9)
xlabel("munki gray Ys")
ylabel("camera gray RGBs")
title("original grayscale Y to RGB relationship")
```



step 6 - linearize camera's RGB response w.r.t relative luminance (Y)

```
% fit polynomial functions between the camera-captured gray-patch RGBs and
% the ColorMunk-measured gray-patch Ys
cam_polys(r, :) = polyfit(cam_gray_rgbs(r, :), munki_gray_Ys, 3);
cam_polys(g, :) = polyfit(cam_gray_rgbs(g, :), munki_gray_Ys, 3);
cam_polys(b, :) = polyfit(cam_gray_rgbs(b, :), munki_gray_Ys, 3);

% use the polynomial functions to linearize the camera's responses to the
% ColorChecker's 24 patches
cam_RSs(r, :) = polyval(cam_polys(r, :), cam_rgbs(r, :));
cam_RSs(g, :) = polyval(cam_polys(g, :), cam_rgbs(g, :));
cam_RSs(b, :) = polyval(cam_polys(b, :), cam_rgbs(b, :));

% clip out-of-range values, produced by quantization errors in the
% calculations
cam_RSs(cam_RSs < 0) = 0;
cam_RSs(cam_RSs > 1) = 1;
```

step 7 - plot measured gray-patch Ys vs linearized camera gray-patch RGBs

```
% extract gray-patch linear RGBs
cam_gray_RSs = flipplr(cam_RSs(:, 19:end));

figure;
hold on

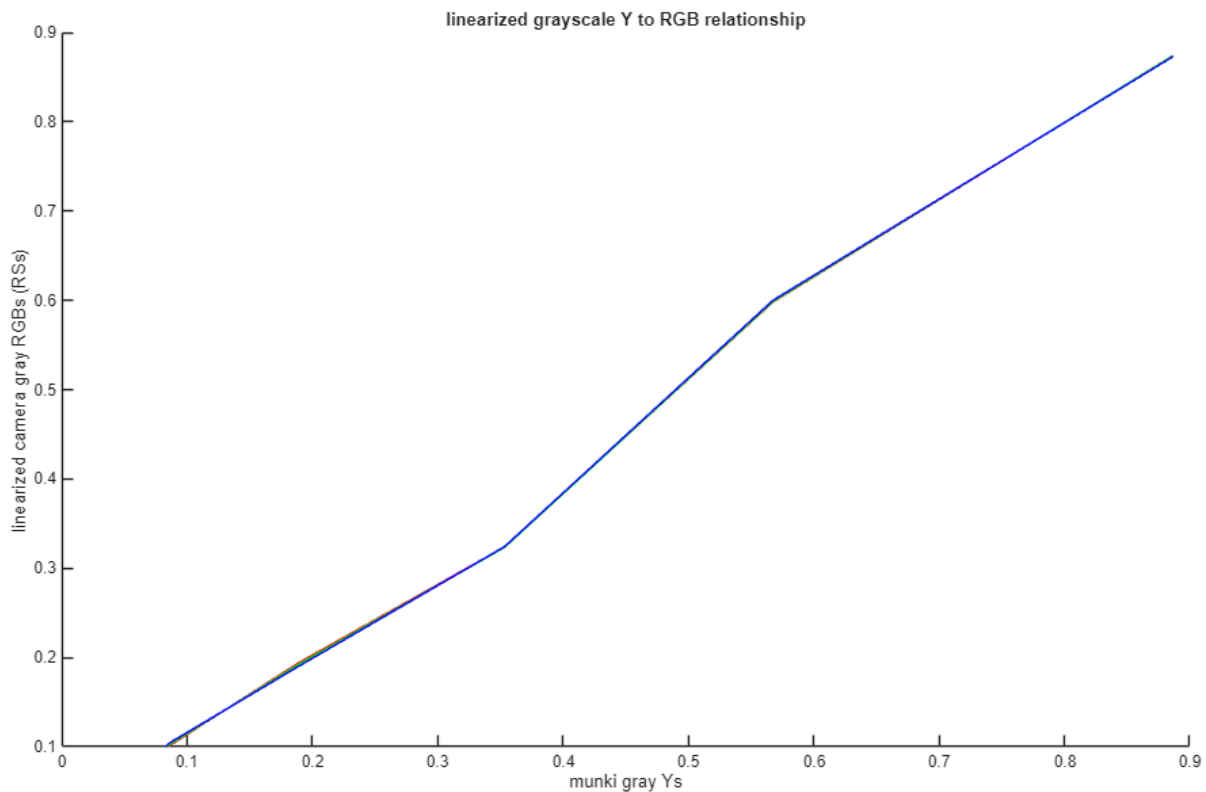
% Ys vs Rs
plot(munki_gray_Ys, cam_gray_RSs(r, :), "Color", 'r');
% Ys vs Gs
plot(munki_gray_Ys, cam_gray_RSs(g, :), "Color", 'g');
% Ys vs Bs
plot(munki_gray_Ys, cam_gray_RSs(b, :), "Color", 'b');

% adjust font size
fontsize(gcf, scale=0.9)

hold off

xlim([0 0.9])
ylim([0.1 0.9])
xticks(0:0.1:0.9)
yticks(0:0.1:0.9)
xlabel("munki gray Ys")
```

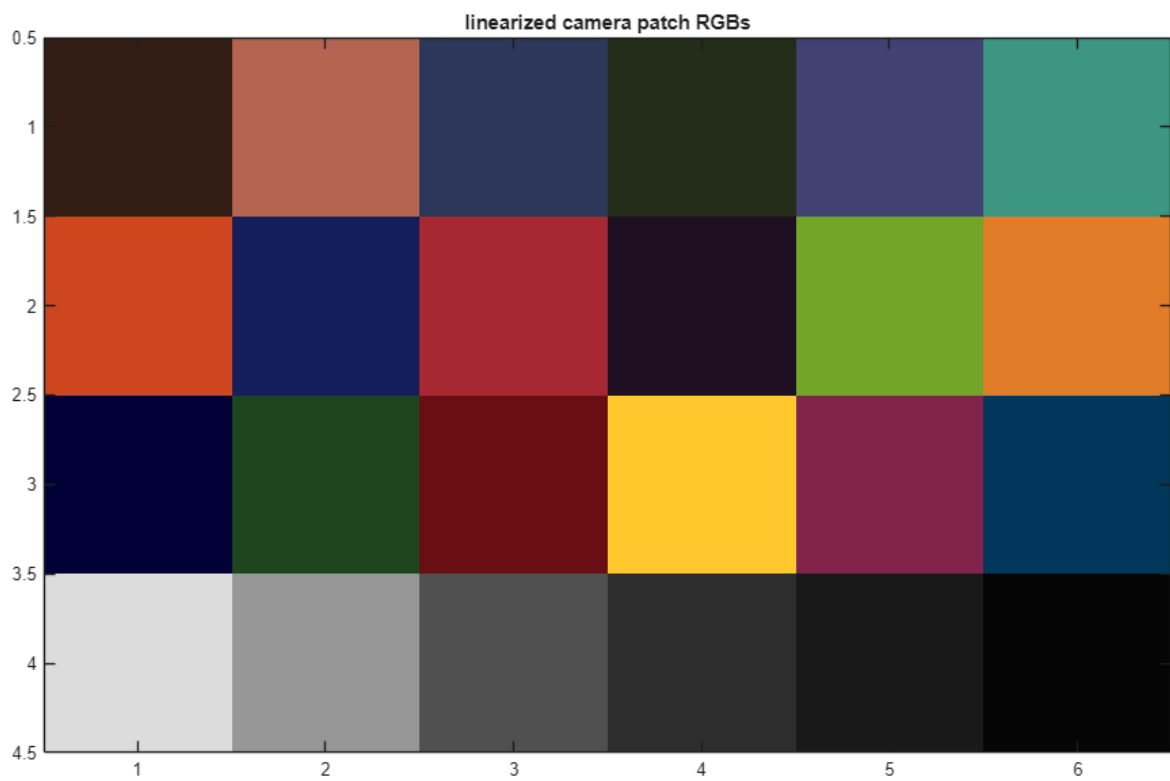
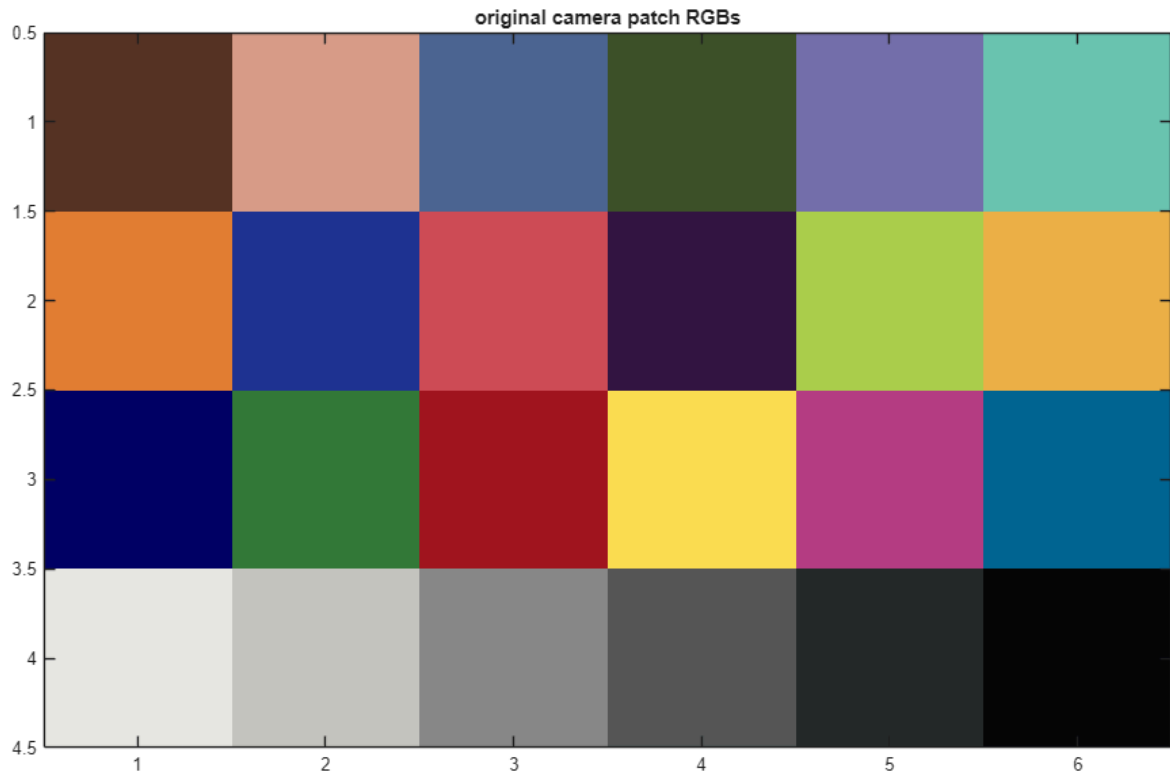
```
ylabel("linearized camera gray RGBs (RSs)")
title("linearized grayscale Y to RGB relationship")
```



step 8 - visualize the pre- and post- linearization ColorChecker patches

```
% visualize the original camera RGBs
pix = reshape(cam_rgbs', [6 4 3]);
pix = uint8(pix * 255);
pix = imrotate(pix, -90);
pix = fliplr(pix);
figure;
image(pix);
title("original camera patch RGBs");

% visualize the linearized camera RGBs
pix = reshape(cam_RSs', [6 4 3]);
pix = uint8(pix * 255);
pix = imrotate(pix, -90);
pix = fliplr(pix);
figure;
image(pix);
title("linearized camera patch RGBs");
```



step 9 - derive a matrix to transform from linearized RGBs to XYZs

```
% use the munki-measured ColorChecker XYZs and camera-captured RGB RSs to
% derive a 3x3 matrix that can be used to estimate XYZs from camera RGBs
cam_matrix3x3 = munki_XYZs * pinv(cam_RSs)
```

```
cam_matrix3x3 =  
  
37.9178    23.0464    26.1710  
14.1415    56.8200    19.8546  
-5.4882    -4.3764    86.7760
```

step 10 - use the matrix to estimate XYZ values from linearized RGBs

```
% derive the estimate XYZs from basic 3x3 transformation matrix  
cam_XYZs = cam_matrix3x3 * cam_RSs
```

```
cam_XYZs =  
  
Columns 1 through 7  
  
12.8532    45.1580    21.4675    12.3560    28.2220    36.1040    40.3298  
11.6239    39.5678    22.5643    14.3711    27.4573    47.1834    30.0361  
6.5118     22.7141    29.3175     7.0814    37.5084    40.3141     4.9187  
  
Columns 8 through 14  
  
15.4292    33.6626     9.9724    36.7352    49.3640     6.8847    14.7523  
15.2389    22.7708     8.0649    47.0999    43.7255     5.5575    20.4136  
29.9524    12.5524    11.8535     9.3367     6.9546    19.4782     9.1576  
  
Columns 15 through 21  
  
19.4533    60.8262    30.6357    14.8118    76.1373    52.1731    28.2130  
11.0143    62.5996    21.3133    19.6591    79.3557    54.3851    29.3912  
4.2764     6.6374    22.9642    29.5968    67.1435    46.1511    24.9354  
  
Columns 22 through 24  
  
16.7522     8.9231     1.9085  
17.4328     9.3215     1.9984  
14.5896     8.0291     1.6569
```

step 11 - evaluate the accuracy of the camera color model

```
% use the XYZ2Lab function to calculate Lab values from the estimated XYZ  
% values  
cie = loadCIEdata;  
% use D50 as reference illuminate  
XYZn_D50 = ref2XYZ(cie.PRD, cie.cmf2deg, cie.illD50);  
cam_Labs = XYZ2Lab(cam_XYZs, XYZn_D50);  
cam_deltaE = deltaEab(munki_Labs, cam_Labs);  
  
% print basic camera model error  
print_camera_model_error(munki_Labs, cam_Labs, cam_deltaE)
```

```
Camera model color error  
camera->camera_RGBs->camera_model->estimated_XYZs
```

colormunki measured vs. camera estimated ColorChecker Lab values								
patch #	measured			estimated			dEab	
	L	a	b	L	a	b		
1	37.1865	14.9985	15.2592	40.6120	11.4016	11.8249	6.0388	
2	65.8188	16.8695	18.0267	69.1606	21.2227	16.7334	5.6383	
3	49.9949	-3.1841	-23.5159	54.6207	-1.3527	-19.8865	6.1583	
4	42.6411	-15.3251	20.0423	44.7604	-9.8161	16.5457	6.8605	
5	54.6852	9.6978	-26.7126	59.3953	7.0004	-23.7783	6.1702	
6	71.2441	-33.1391	-0.5010	74.3069	-28.8682	-1.8109	5.4164	
7	62.2558	34.1094	57.7774	61.6854	39.0801	55.8173	5.3735	
8	39.5890	9.9980	-43.6388	45.9596	4.3875	-35.8345	11.5312	
9	51.8424	48.1403	16.0636	54.8355	46.7469	15.3711	3.3734	
10	29.4495	22.4255	-21.7661	34.1177	18.6769	-18.3300	6.9030	
11	71.6264	-24.3441	57.6850	74.2536	-26.5503	58.8799	3.6328	
12	72.2288	20.6039	69.0149	72.0446	20.4907	64.1173	4.9024	
13	28.6402	18.5907	-51.4092	28.2676	16.6242	-47.2749	4.5933	
14	54.6309	-39.5493	32.8341	52.3016	-26.9783	21.6542	16.9837	
15	42.5988	54.6049	25.7315	39.6046	53.5810	21.3074	5.4393	
16	82.4265	3.8689	78.8570	83.2312	1.1037	84.7581	6.5663	
17	51.5476	49.5154	-14.3758	53.2906	42.5201	-11.1044	7.9167	
18	49.3892	-26.5473	-28.6645	51.4496	-22.9468	-25.8022	5.0400	
19	95.4458	-0.4414	0.0244	91.3950	-0.7605	-1.5447	4.3558	
20	80.0339	0.1309	-0.9345	78.6857	-0.6870	-1.5233	1.6832	
21	66.0107	-0.0004	-1.1463	61.1254	-0.4922	-1.2300	4.9107	

22	50.5546	-0.6207	-0.9616	48.8007	-0.3118	-0.5223	1.8343
23	35.1532	-0.0632	-0.9708	36.5960	-0.5443	-1.3026	1.5567
24	20.3224	-0.2858	-0.5603	15.4786	-0.4299	-0.0840	4.8693
					min		1.5567
					max		16.9837
					mean		5.7395

step 12 - compensate for interactions and non-linearities

```
% create a vector to represent the original set of radiometric scalars used
% to derive the 3x3 matrix plus additional terms that represent the
% products of the individual RGB channels (interactions) and squares of the
% individual RGB channels (non-linearities)

% split the radiometric scalars (cam_RSs) into r, g, b vectors
RSrgbs = cam_RSs;
RSrs = RSrgbs(r, :);
RSgs = RSrgbs(g, :);
RSbs = RSrgbs(b, :);

% create vectors of these RSs with multiplicative terms to represent
% interactions, and square terms to represent non-linearities in the
% RGB-to-XYZ relationship
RSrgbs_extd = [RSrgbs; RSrs .* RSgs; RSrs .* RSbs; RSgs .* RSbs;
    RSrs .* RSgs .* RSbs; RSrs .^2; RSgs .^2; RSbs .^2;
    ones(1, size(RSrgbs, 2))];

% find the extended (3x11) matrix that relates the RS and XYZ datasets
cam_matrix3x11 = munki_XYZs * pinv(RSrgbs_extd)
```

cam_matrix3x11 =

Columns 1 through 7

60.3684	10.5014	28.0459	43.1533	-50.3516	39.4986	57.2943
26.7739	56.8122	22.2698	31.1110	-42.1926	50.5412	66.5172
3.2647	-16.0896	105.3158	20.2386	-24.2594	16.2095	55.3957

Columns 8 through 11

-31.8335	-29.9785	-20.9268	-0.7296
-19.8788	-42.9709	-38.7108	-0.6113
-15.0706	-9.2005	-44.6557	-1.1490

step 13 - estimate XYZs from the extended matrix and RS representation

```
% estimate XYZs from the RSs using the extended matrix and RS representation
cam_XYZs = cam_matrix3x11 * RSrgbs_extd
```

cam_XYZs =

Columns 1 through 7

14.0473	43.8320	19.8797	12.2249	25.6412	32.5554	38.8698
12.6876	39.4648	20.8470	14.9245	23.7956	43.8960	29.5435
6.6424	22.6722	28.3970	6.4942	35.1063	36.9023	2.9293

Columns 8 through 14

13.2682	31.6252	10.3254	34.4290	48.5358	5.6363	12.7895
12.3413	20.9748	8.0523	43.8645	43.8424	3.7260	19.9212
29.1911	11.2957	12.5054	8.0043	6.0540	20.1760	7.6677

Columns 15 through 21

21.4157	60.5363	27.5580	11.2514	85.4257	52.7015	27.9510
11.7656	60.7092	17.5698	17.0508	88.8678	54.7715	29.3904
4.4807	7.8557	21.6169	27.7920	73.2764	45.8516	24.7603

Columns 22 through 24

16.8614	8.9073	1.4112
17.9166	9.6539	1.6877
14.7354	7.9475	0.8151

step 14 - evaluate the extended camera model by using deltaE

```

cam_Labs = XYZ2Lab(cam_XYZs, XYZn_D50);
cam_deltaE = deltaEab(munki_Labs, cam_Labs);

% print extended camera model error
print_camera_model_error(munki_Labs, cam_Labs, cam_deltaE)

```

```

Camera model color error
camera->camera_RGBs->camera_model->estimated_XYZs

```

patch #	measured			estimated			dEab
	L	a	b	L	a	b	
1	37.1865	14.9985	15.2592	42.2887	11.8511	14.1459	6.0974
2	65.8188	16.8695	18.0267	69.0866	17.7032	16.6859	3.6293
3	49.9949	-3.1841	-23.5159	52.7816	-1.0888	-21.5590	3.9982
4	42.6411	-15.3251	20.0423	45.5306	-14.0307	20.3828	3.1844
5	54.6852	9.6978	-26.7126	55.8826	11.6969	-26.4795	2.3419
6	71.2441	-33.1391	-0.5010	72.1589	-31.8263	-0.9394	1.6590
7	62.2558	34.1094	57.7774	61.2583	36.3532	67.4762	10.0048
8	39.5890	9.9980	-43.6388	41.7535	9.2011	-41.8673	2.9083
9	51.8424	48.1403	16.0636	52.9219	47.7440	15.7605	1.1892
10	29.4495	22.4255	-21.7661	34.0915	21.5269	-20.2611	4.9619
11	71.6264	-24.3441	57.6850	72.1379	-25.1802	60.0715	2.5799
12	72.2288	20.6039	69.0149	72.1230	17.9033	68.2133	2.8190
13	28.6402	18.5907	-51.4092	22.7441	27.0494	-58.2567	12.3775
14	54.6309	-39.5493	32.8341	51.7480	-37.0210	26.2231	7.6426
15	42.5988	54.6049	25.7315	40.8412	57.7979	22.2705	5.0263
16	82.4265	3.8689	78.8570	82.2221	4.7708	78.0302	1.2405
17	51.5476	49.5154	-14.3758	48.9700	49.3123	-15.9484	3.0263
18	49.3892	-26.5473	-28.6645	48.3239	-32.9240	-28.2423	6.4788
19	95.4458	-0.4414	0.0244	95.5251	-0.4859	0.0563	0.0964
20	80.0339	0.1309	-0.9345	78.9095	-0.2805	-0.7803	1.2072
21	66.0107	-0.0004	-1.1463	61.1246	-1.5196	-0.9165	5.1220
22	50.5546	-0.6207	-0.9616	49.3948	-2.2675	0.1293	2.2907
23	35.1532	-0.0632	-0.9708	37.2139	-3.3412	0.0757	4.0109
24	20.3224	-0.2858	-0.5603	13.7550	-5.9489	8.3907	12.4629
min							0.0964
max							12.4629
mean							4.4315

step 15 - save the extended camera model

```

% save the (extended) camera model for use in later projects
save(fullfile(paths.p05.data, 'cam_model.mat'), ...
    "cam_polys", "cam_matrix3x11");

```

step 16 - create camRGB2XYZ function to convert camera RGBs to estimated XYZs

create a function that converts camera-captured RGBs to XYZs based on the camera model derived above

```

function cameraXYZ = camRGB2XYZ(cameraModelFile, cameraRGB)
%CAMRGB2XYZ Estimate scene XYZ values from iPhone SE camera RGB values.
% CAMERAXYZ = CAMRGB2XYZ(CAMERAMODELFILE, CAMERARGB) loads the Project 5
% model and converts a 3-by-N matrix of 8-bit camera RGB values to a
% 3-by-N matrix of estimated CIE XYZ values.

RED = 1;
GREEN = 2;
BLUE = 3;

cameraModel = load(cameraModelFile);
normalizedRGB = double(cameraRGB) ./ 255;
linearRGB = zeros(size(normalizedRGB));

for channel = 1:3
    linearRGB(channel, :) = polyval( ...
        cameraModel.cam_polys(channel, :), normalizedRGB(channel, :));
end

% Match the clipping used when the characterization model was fitted.
linearRGB = min(max(linearRGB, 0), 1);

red = linearRGB(RED, :);
green = linearRGB(GREEN, :);
blue = linearRGB(BLUE, :);

extendedRGB = [linearRGB;
    red .* green;
    red .* blue;
    green .* blue;

```



```

        red .* green .* blue;
        red .^ 2;
        green .^ 2;
        blue .^ 2;
        ones(1, size(linearRGB, 2))];

cameraXYZ = cameraModel.cam_matrix3x11 * extendedRGB;
end

```

```

% test the function by using it to estimate XYZs from the cam_RGBs'
cam_XYZs = camRGB2XYZ(fullfile(paths.p05.data, 'cam_model.mat'), ...
    cam_RGBs)

```

```

cam_XYZs =

Columns 1 through 7

    14.0473    43.8320    19.8797    12.2249    25.6412    32.5554    38.8698
    12.6876    39.4648    20.8470    14.9245    23.7956    43.8960    29.5435
     6.6424    22.6722    28.3970     6.4942    35.1063    36.9023     2.9293

Columns 8 through 14

    13.2682    31.6252    10.3254    34.4290    48.5358     5.6363    12.7895
    12.3413    20.9748     8.0523    43.8645    43.8424     3.7260    19.9212
    29.1911    11.2957    12.5054     8.0043     6.0540    20.1760     7.6677

Columns 15 through 21

    21.4157    60.5363    27.5580    11.2514    85.4257    52.7015    27.9510
    11.7656    60.7092    17.5698    17.0508    88.8678    54.7715    29.3904
     4.4807     7.8557    21.6169    27.7920    73.2764    45.8516    24.7603

Columns 22 through 24

    16.8614     8.9073     1.4112
    17.9166     9.6539     1.6877
    14.7354     7.9475     0.8151

```

step 17 - visualize camera model and camRGB2XYZ function

```

XYZn_D65 = ref2XYZ(cie.PRD, cie.cmf2deg, cie.illD65);

% visualize the munki-measured XYZs as an sRGB image
munki_XYZs_D65 = catBradford(munki_XYZs, XYZn_D50, XYZn_D65);
munki_XYZs_sRGBs = XYZ2sRGB(munki_XYZs_D65);
pix = reshape(munki_XYZs_sRGBs', [6 4 3]);
pix = uint8(pix*255);
pix = imrotate(pix, -90);
pix = fliplr(pix);
figure;
image(pix);
title("munki XYZs chromatically adapted and visualized in sRGB");

% visualize the camera-estimated XYZs as an sRGB image
cam_XYZs_D65 = catBradford(cam_XYZs, XYZn_D50, XYZn_D65);
cam_XYZs_sRGBs = XYZ2sRGB(cam_XYZs_D65);
pix = reshape(cam_XYZs_sRGBs', [6 4 3]);
pix = uint8(pix*255);
pix = imrotate(pix, -90);
pix = fliplr(pix);
figure;
image(pix);
title("estimated XYZs chromatically adapted and visualized in sRGB");

```

